

CSCI 1202 — Programming Principles II

Spring 2026 | ADA University

The Skyways

Airport Dispatch Tycoon — Term Project Report

Team Number: 6

Submission Deadline: May 4, 2026

GitHub Repository: [Team6](#)

3.1 Team Information & Work Allocation

Full Name	Student ID	% of Work	What Was Done
Tural Gasimov	16051	~50%	- OOP - Aircraft: CargoFreighter, CommercialJet, PrivateCharter DepotManager, SupplyItem IGroundService: BaggageHandler, FuelingTrack, CateringVan - Report
Faiq Tayibov	20472	~50%	- GUI - AirportDashboard - Persistence- SystemLogger, SaveLoad mechanism - Control/Management- AirportController, FlightQueueManager - Report
Emil Hajiyeu	18043	~0%	
Shamil Aliyev	17546	~0%	

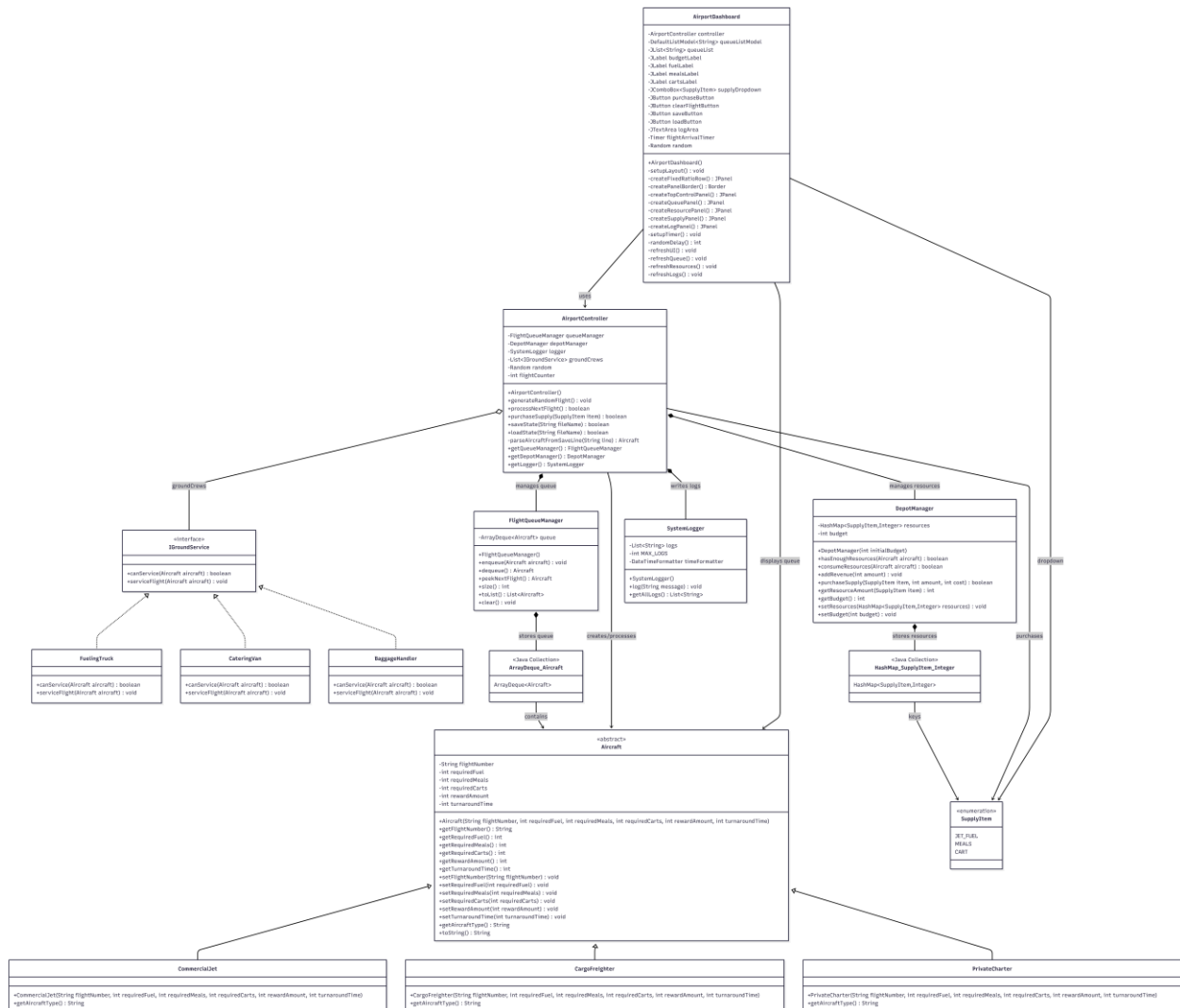
3.2 System Overview

Skyways Airport Dispatch Dashboard is a real-time airport ground operations simulation built in Java using Swing. The system automatically generates incoming aircraft every 2-5 seconds and adds them to ArrayDeque queue. Each aircraft is represented by the abstract Aircraft class and one of its subclasses, such as CommercialJet, CargoFreighter, or PrivateCharter. The user can poll the next flight from the queue by pressing the “Clear Next Flight” button. Before a flight is cleared, the system checks the DepotManager to make sure there is enough jet fuel, meals, and luggage carts. If enough resources are available, the correct ground service crews process the aircraft polymorphically through the IGroundService interface. After successful processing, resources are deducted, airport budget revenue is added, and the event is written to the system log. If resources are insufficient, the flight is not polled from the queue, and an error message is logged with no state change. The dashboard also allows the user to purchase supplies and save or load the full airport state, including budget, resources, flight counter, and pending queue through SaveLoad mechanism.

3.3 UML Class Diagram

Create your diagram using draw.io at diagrams.net (free). Export as PNG and insert the image below. Your diagram must show:

- All main classes with key attributes and methods
- Aircraft → CommercialJet, CargoFreighter, PrivateCharter
- IGroundService → FuelingTruck, CateringVan, BaggageHandler
- DepotManager uses HashMap<SupplyItem, Integer>
- FlightQueueManager uses ArrayDeque<Aircraft>
- AirportController coordinates flight generation, queue processing, resource validation, purchasing, logging, save, and load.
- AirportDashboard displays the GUI and interacts with AirportController.



Note: I will submit a pdf version of the diagram in case of resolution issues.

3.4 Key Design Decisions

How Tasks Are Structured

In Skyways, tasks are structured as incoming aircraft that need ground service. We created an abstract Aircraft class that stores shared task data such as flight number, required fuel, required meals, required carts, reward amount, and turnaround time. The concrete aircraft types CommercialJet, CargoFreighter, and PrivateCharter extend Aircraft and override getAircraftType() to identify the specific flight type. New aircraft objects are randomly generated by the AirportController and added to the flight queue. Each aircraft type has different resource requirements, so processing a flight may require fuel, meals, luggage carts, or a combination of them. These required values are later used by the depot and ground service classes to decide whether the flight can be cleared successfully.

How Processors Are Selected

In Skyways, processors are represented by ground service crew classes. The IGroundService interface defines two methods: canService(Aircraft aircraft) and serviceFlight(Aircraft aircraft). The concrete processor classes are FuelingTruck, CateringVan, and BaggageHandler. The AirportController stores these processors in a List<IGroundService> and loops through the list whenever a flight is cleared. Each processor decides whether it can service the aircraft by checking the aircraft's required resources. For example, the fueling truck checks fuel requirements, the catering van checks meal requirements, and the baggage handler checks luggage cart requirements. If canService() returns true, the controller calls serviceFlight() on that processor. This allows processor selection to happen polymorphically through the interface without using instanceof checks or hardcoded aircraft-type conditions.

How Resources Are Validated and Consumed

In Skyways, resources are managed inside the DepotManager class using a private HashMap<SupplyItem, Integer>. The map stores the current amounts of jet fuel, meals, and luggage carts, while the budget is stored separately as an integer. When the user clicks "Clear Next Flight," the AirportController first looks at the next aircraft in the queue without removing it. Before any processing happens, DepotManager.hasEnoughResources() checks whether there is enough fuel, meals, and carts for that aircraft. If the resources are insufficient, the flight stays in the queue, no resources are deducted, and an error message is added to the system log. If enough resources are available, consumeResources() subtracts the required amounts from the map. Since the resource check happens before consumption, resource values cannot become negative. After successful processing, the airport budget is increased by the aircraft's reward amount. Purchasing supplies also goes through DepotManager.purchaseSupply(), which checks whether the budget is enough before adding supplies and subtracting the purchase cost.

How Save / Load Works

In Skyways, save and load functionality is handled inside the AirportController. When the user clicks "Save State," the system writes the current airport state to saveAndLoad/airport_state.txt. The saved file includes the airport budget, the flight counter, all resource amounts, and the pending flight queue. Resources are saved line by line using the format RESOURCE, item,

amount, and queued aircraft are saved between QUEUE_START and QUEUE_END. Each aircraft line stores the aircraft subclass type, flight number, required fuel, required meals, required carts, reward amount, and turnaround time. When loading, the system reads the file line by line and restores the saved budget, resources, and flight counter. The current queue is cleared first, then the saved aircraft are rebuilt and added back in the same order. Aircraft objects are reconstructed using a switch statement in parseAircraftFromSaveLine(), which creates the correct subclass such as CommercialJet, CargoFreighter, or PrivateCharter. This allows the simulation to continue from the saved state instead of mixing old runtime data with loaded data.

Take screenshots of your running application and insert them into the boxes below.

